SWE30009 SOFTWARE TESTING AND RELIABILITY

Metamorphic Testing & Mutation Analysis of Manhattan Distance

A Complete Evaluation: 5W1H Framework, Proportional Sampling, and E/P-Measures

Student Name: [Your Full Name] | **Student ID: [Your Student ID]**

Unit: SWE30009 — Software Testing and Reliability (Semester 2, 2026)

Department of Computing Technologies, Swinburne University of Technology

TESTING CONCEPTS (5W1H FRAMEWORK)

Task 1.1: The Test Oracle Problem & Untestable Systems

WHAT is a Test Oracle?

A mechanism, specification, or reference to verify whether actual output matches expected golden output (Barr et al., 2015).

WHO & WHERE does it apply?

Used by testers and CI harnesses. Pervasive across untestable systems: AI models, search engines, and scientific computing.

WHEN does the Problem arise?

Arises during test assessment when expected output cannot be pre-computed, verified, or written as 'assert actual == expected'.

WHY is it Critical?

Without an oracle, traditional point-wise testing completely breaks down. Testing cannot judge if an output is correct.

HOW do Untestable Systems arise?

Occurs when: (1) No oracle exists (machine learning, astrophysics); or (2) Oracle verification is computationally prohibitive (NP-hard).

THE SOLUTION: Metamorphic Testing

Instead of checking a single output, MT tests relational invariants across multiple executions without needing a golden oracle.

JUSTIFICATION + WHY (TREATING READER AS A BEGINNER)

Justification: If testing ceases simply because no golden oracle exists, safety-critical software (such as autonomous vehicle navigation, radiation therapy machines, or aircraft avionics) would be deployed with undetected critical faults.
Why: Real-world algorithms have inherent symmetries, monotonicity, additive invariants, or geometric properties. Even when an oracle cannot verify individual values, it can effortlessly verify the consistency of relationships between related executions (Weyuker, 1982; Chen et al., 2018).

CORE CONCEPT & ANALOGY FOR BEGINNERS

Task 1.1: Metamorphic Testing (MT) Methodology & Intuition

Intuition: Beginner Trigonometry Analogy

- Imagine testing a library computing sin(x) for x = 138.4912°.
- Without a calculator, you cannot know sin(138.4912°) in your head.
- However, you know geometry: sin(x) == sin(180° - x).
- Execution 1: Program(138.4912°) -> Result 1
- Execution 2: Program(180° - 138.4912° = 41.5088°) -> Result 2
- Check: Does Result 1 == Result 2?
- If they differ, a bug is 100% guaranteed — even though you never knew the true decimal answer!

Metamorphic Relations & Test Groups

- Metamorphic Relation (MR): A necessary invariant property over distinct inputs and outputs across multiple program executions.
- Metamorphic Test Group (MTG):
- • Source Test Case (ts): Baseline input fed to SUT -> Output O1.
- • Follow-up Test Case (tf): Transformed input created via MR rule -> Output O2.
- MR Verification: Assert MR_Condition(O1, O2).
- Why Multiple MTGs: Prevents fortuitous error cancellation (e.g., accidental equality via 0 multiplication) across domain boundaries.

EFFECTIVENESS IN REAL-WORLD SOFTWARE

Justification: Metamorphic testing exposes logical and algorithmic faults that pass traditional unit tests because it tests consistency across execution boundaries.
Why: Proven across compilers (GCC/LLVM bug detection via EMI, Le et al., 2014), search engines (ranking invariance under query word permutations), and autonomous driving (image perspective and brightness invariance).

Task 1.2: Proportional Sampling Strategy Algorithm

Algorithmic Principles & Formula

- Concept: Divides input domain D into k disjoint partitions D1, ..., Dk of sizes N1, ..., Nk (Total domain N = sum(Ni)).
- Proportional Allocation Formula: ni = n * (Ni / N)
- Given Problem Parameters (N = 550, m = 10 failure inputs):
- • D1: Size N1 = 100, m1 = 3 failure inputs (Failure rate θ1 = 0.030)
- • D2: Size N2 = 200, m2 = 5 failure inputs (Failure rate θ2 = 0.025)
- • D3: Size N3 = 250, m3 = 2 failure inputs (Failure rate θ3 = 0.008)
- Total Domain Size: N = 100 + 200 + 250 = 550 inputs.

Fault Detection Probability Formulation

- Under random sampling with replacement within each partition:
- Single test non-detection prob: (1 - θi)
- All ni tests fail to detect failure: (1 - θi)^{ni}
- Overall detection of at least one failure across all partitions:
- P(detect) = 1 - product_{i=1}^k (1 - θi)^{ni}
- Key Observation: D1 has highest failure density (3%), but smallest domain share (18.18%). D3 has lowest failure density (0.8%), but largest domain share (45.45%).

JUSTIFICATION + WHY (PARTITION STRATEGY)

Justification: Allocating test cases proportionally ensures representative coverage of the operational profile.
Why: Large partitions represent most real-world usage. Proportional sampling avoids starving large partitions while ensuring smaller, error-dense partitions receive systematic testing (Chen et al., 2004).

STEP-BY-STEP PROBABILITY EVALUATION

Task 1.2: Mathematical Calculations (Scenario A vs B)

Scenario A: Budget n = 5 Test Cases

- Step 1: Allocation Calculation ni = 5 * (Ni / 550)
- • n1 = 5 * (100/550) = 0.909 -> 1 test case
- • n2 = 5 * (200/550) = 1.818 -> 2 test cases
- • n3 = 5 * (250/550) = 2.273 -> 2 test cases
- • Budget Check: n1 + n2 + n3 = 1 + 2 + 2 = 5 test cases
- Step 2: Non-detection Probabilities (1 - θi)^{ni}
- • D1: (1 - 0.030)^1 = 0.97000
- • D2: (1 - 0.025)^2 = 0.95063
- • D3: (1 - 0.008)^2 = 0.98406
- Step 3: Overall Detection Probability
- • Product = 0.97000 * 0.95063 * 0.98406 = 0.90740
- • P(detect) A = 1 - 0.90740 = 0.09260 -> 9.26%

Scenario B: Budget n = 10 Test Cases

- Step 1: Allocation Calculation ni = 10 * (Ni / 550)
- • n1 = 10 * (100/550) = 1.818 -> 2 test cases (Remainder 0.818)
- • n2 = 10 * (200/550) = 3.636 -> 4 test cases (Remainder 0.636)
- • n3 = 10 * (250/550) = 4.545 -> 4 test cases (Remainder 0.545)
- • Budget Check (Hamilton Method): 2 + 4 + 4 = 10 test cases
- Step 2: Non-detection Probabilities (1 - θi)^{ni}
- • D1: (1 - 0.030)^2 = 0.94090
- • D2: (1 - 0.025)^4 = 0.90369
- • D3: (1 - 0.008)^4 = 0.96838
- Step 3: Overall Detection Probability
- • Product = 0.94090 * 0.90369 * 0.96838 = 0.82339
- • P(detect) B = 1 - 0.82339 = 0.17661 -> 17.66%

COMPARATIVE INSIGHT (JUSTIFICATION + WHY)

Justification: Doubling the test budget from n = 5 to n = 10 nearly doubles fault detection likelihood from 9.26% to 17.66%.
Why: Adding test cases to D1 and D2 leverages their higher failure rates (3% and 2.5%), significantly compounding detection probability across the domain.

Task 2.1: Program Under Test (SUT) Selection

SUT Provenance & Specification

- Repository: Public GitHub (TheAlgorithms/Python)
- File: maths/manhattan_distance.py
- Mathematical Definition (L1 Norm Distance):
- d_M(u, v) = sum_{i=1}^n |u_i - v_i|
- Author: TheAlgorithms Community (Public Open Source)
- Language: Python 3 (Pure standard library)
- Why Selected (Justification + Why):
- • Clean mathematical invariants (symmetry, shift invariance).
- • Contains rich syntactic elements: type assertions, length checks, loops, abs(), subtraction, and sum().
- • Perfect for creating >= 30 diverse non-equivalent mutants.
- • Original program passed 100% of defined MTGs.

Test Input Vectors (2D Coordinate Points)

- Vectors formatted strictly as required:
- T1 = [12, 34]
- T2 = [5, 10]
- T3 = [-3, 8]
- T4 = [0, 0]
- T5 = [100, 250]
- Why 2D Vectors (Beginner-Friendly Justification):
- • 2D points (x, y) represent intuitive positions on a grid map (like navigating city streets).
- • Eliminates unnecessary multidimensional complexity while fully testing all loops and conditions.
- • Easily verified by hand for crystal-clear explanations.

Task 2.2: Formulation of 2 Metamorphic Relations

MR1: Symmetry (Commutativity)

- Mathematical Property: Distance from A to B must equal distance from B to A in metric space.
- Formula: manhattan_distance(A, B) == manhattan_distance(B, A)
- Proof: |u_i - v_i| == |-(v_i - u_i)| == |v_i - u_i|
- Test Construction:
- • Source Case (ts): Pass (TA, TB) -> Output O1
- • Follow-up Case (tf): Swap arguments (TB, TA) -> Output O2
- • Verification: Assert O1 == O2
- Targeted Faults: Asymmetric operations, directional filtering, missing abs() calls.

MR2: Translation / Shift Invariance

- Mathematical Property: Translating both points along coordinate axes by offset c preserves distance.
- Formula: manhattan_distance(A, B) == manhattan_distance(A + c, B + c)
- Proof: |(u_i + c) - (v_i + c)| = |u_i - v_i + c - c| = |u_i - v_i|
- Test Construction:
- • Source Case (ts): Pass (TA, TB) -> Output O1
- • Follow-up Case (tf): Shift both points by c -> Output O2
- • Verification: Assert O1 == O2
- Targeted Faults: Operator swaps (a+b), coordinate scaling, non-invariant multiplications, type restrictions.

5 MTGS FOR MR1 (SYMMETRY) + 5 MTGS FOR MR2 (SHIFT INVARIANCE)

Task 2.3: 10 Metamorphic Test Groups (MTGs)

MTG	MR Type	Source Inputs (ts)	Transformation / Shift	Expected Metamorphic Relation
MTG 1.1	MR1 (Symmetry)	(T1, T2) = ([12,34], [5,10])	Swap: (T2, T1)	dist(T1, T2) == dist(T2, T1)
MTG 1.2	MR1 (Symmetry)	(T1, T3) = ([12,34], [-3,8])	Swap: (T3, T1)	dist(T1, T3) == dist(T3, T1)
MTG 1.3	MR1 (Symmetry)	(T2, T3) = ([5,10], [-3,8])	Swap: (T3, T2)	dist(T2, T3) == dist(T3, T2)
MTG 1.4	MR1 (Symmetry)	(T1, T4) = ([12,34], [0,0])	Swap: (T4, T1)	dist(T1, T4) == dist(T4, T1)
MTG 1.5	MR1 (Symmetry)	(T3, T5) = ([-3,8], [100,250])	Swap: (T5, T3)	dist(T3, T5) == dist(T5, T3)
MTG 2.1	MR2 (Shift)	(T1, T2) = ([12,34], [5,10])	Shift offset c = +10	dist(T1, T2) == dist(T1+10, T2+10)
MTG 2.2	MR2 (Shift)	(T1, T3) = ([12,34], [-3,8])	Shift offset c = -25	dist(T1, T3) == dist(T1-25, T3-25)
MTG 2.3	MR2 (Shift)	(T2, T4) = ([5,10], [0,0])	Shift offset c = +100	dist(T2, T4) == dist(T2+100, T4+100)
MTG 2.4	MR2 (Shift)	(T3, T5) = ([-3,8], [100,250])	Shift offset c = +3.5 (Float)	dist(T3, T5) == dist(T3+3.5, T5+3.5)
MTG 2.5	MR2 (Shift)	(T4, T5) = ([0,0], [100,250])	Shift offset c = -50	dist(T4, T5) == dist(T4-50, T5-50)

STANDARD MUTATION OPERATORS TAXONOMY

Task 2.4: 30 Non-Equivalent Mutants (MUTANTS/)

Arithmetic & Operator Mutants

- AOR (Arithmetic Operator Replacement - 15 Mutants):
- • M1: (a + b) instead of (a - b) [Addition swap]
- • M4: Truncate to int; M5: Chebyshev max()
- • M8: +1.0 bias; M9: -1.0 bias; M10: *2.0 scale
- • M12: (a-b)^2 [Squared difference]
- • M14: +0.5 coordinate bias; M17: abs(a * b)
- • M18: abs(a) + abs(b); M20: Mean Absolute Error
- • M26: abs(a - 2b); M27: abs(2a - b) [Asymmetric]
- • M28: Floor division abs(a // b)
- • M29/M30: Injected coordinate bias +abs(a), +abs(b)
- UOI (Unary Operator - 1 Mutant):
- • M3: Negated distance -abs(a - b)

Absolute, Relational & Type Mutants

- ABS (Absolute Value Modification - 3 Mutants):
- • M2: Omitted abs() -> raw difference (a - b)
- • M13: abs(a) - abs(b) instead of abs(a - b)
- • M19: a - abs(b) asymmetric difference
- ROR (Relational Operator Replacement - 4 Mutants):
- • M6: Inverted length check: len(a) == len(b)
- • M7: Changed length check to len(a) < len(b)
- • M24: Accumulate only when a > b, else 0
- • M25: Accumulate only when a < b, else 0
- SDL (Statement Deletion - 3 Mutants):
- • M11: Truncated last dimension [:-1]
- • M21/M22: Deleted _validate_point calls
- COR & LCR (Ordering & Type - 3 Mutants):
- • M15/M16: Reversed/sorted vector before zip
- • M23: Restricted type to int only (rejects float)

Killing mutants in metamorphic testing

- **A mutant is said to be killed if the relation of a test group and its outputs violates the MR.**

- SUT Output: Satisfies MR (==) ➔ **PASSED**
- Mutant Output: Violates MR (≠) ➔ **KILLED**

MR1: Symmetry (Commutativity) — Execution Table

manhattan_distance(A, B) == manhattan_distance(B, A)

Source test case	Follow-up test case	Relation	SUT	M1 output (a + b)	M2 output (no abs)	M6 output (== guard)	M13 output (|a| - |b|)	M24 output (a > b only)	M25 output (a < b only)
[12, 34]	[5, 10]	dist(T1,T2) == dist(T2,T1)	31 == 31	61 == 61	**31 =/= -31**	**ERR =/= ERR**	**31 =/= -31**	**31 =/= 0**	**0 =/= 31**
[12, 34]	[-3, 8]	dist(T1,T3) == dist(T3,T1)	41 == 41	51 == 51	**41 =/= -41**	**ERR =/= ERR**	**35 =/= -35**	**41 =/= 0**	**0 =/= 41**
[5, 10]	[-3, 8]	dist(T2,T3) == dist(T3,T2)	10 == 10	20 == 20	**10 =/= -10**	**ERR =/= ERR**	**4 =/= -4**	**10 =/= 0**	**0 =/= 10**
[12, 34]	[0, 0]	dist(T1,T4) == dist(T4,T1)	46 == 46	46 == 46	**46 =/= -46**	**ERR =/= ERR**	**46 =/= -46**	**46 =/= 0**	**0 =/= 46**
[-3, 8]	[100, 250]	dist(T3,T5) == dist(T5,T3)	345 == 345	355 == 355	**-345 =/= 345**	**ERR =/= ERR**	**-339 =/= 339**	**0 =/= 345**	**345 =/= 0**

MR1 Summary: 11 / 30 Mutants KILLED (M2, M6, M13, M19, M24, M25, M26, M27, M28, M29, M30). Red '=/' indicates mutant killed. M1 survived because addition (a + b) is symmetric.

MR2: Shift Invariance — Execution Table

manhattan_distance(A, B) == manhattan_distance(A + c, B + c)

Source test case	Follow-up test case	Shift c	SUT	M1 output (a + b)	M2 output (no abs)	M6 output (== guard)	M17 output (a * b)	M18 output (|a| + |b|)	M23 output (int only)
[12, 34]	[22, 44]	c = +10	31 == 31	**61 =/= 101**	31 == 31	**ERR =/= ERR**	**400 =/= 1210**	**61 =/= 101**	31 == 31
[12, 34]	[-13, 9]	c = -25	41 == 41	**51 =/= 49**	41 == 41	**ERR =/= ERR**	**308 =/= 517**	**57 =/= 67**	41 == 41
[5, 10]	[105, 110]	c = +100	15 == 15	**15 =/= 415**	15 == 15	**ERR =/= ERR**	**0 =/= 21500**	**15 =/= 415**	15 == 15
[-3, 8]	[0.5, 11.5]	c = +3.5	345 == 345	**355 =/= 369**	-345 == -345	**ERR =/= ERR**	**2300 =/= 2967**	**361 =/= 369**	**ERR =/= ERR**
[0, 0]	[-50, -50]	c = -50	350 == 350	**350 =/= 150**	-350 == -350	**ERR =/= ERR**	**0 =/= 12500**	350 == 350	350 == 350

MR2 Summary: 12 / 30 Mutants KILLED (M1, M6, M13, M17, M18, M19, M23, M26, M27, M28, M29, M30). Red '=/' indicates mutant killed. M1 is killed because (a+c)+(b+c) != a+b. M23 killed under flo

POWERSHELL TERMINAL CAPTURE OF TEST.PY

Task 2.5: Live Program Execution & Demonstration

PowerShell — python TEST\test.py

```
--- DETAILED METAMORPHIC & MUTATION EXECUTION OUTPUTS (LECTURE SLIDE FORMAT) ---

MR1: SYMMETRY (dist(A, B) == dist(B, A))
MTG      | Source Case         | Follow-up Case      | SUT        | M1 Output  | M2 Output   | M6 Output   | M13 Output  | M24 Output  | M25 Output
-------------------------------------------------------------------------------------------------------------------------------------------
MTG 1.1  | [12, 34], [5, 10]   | [5, 10], [12, 34]   | 31 == 31   | 61 == 61   | 31 =/= -31  | ERR =/= ERR | 31 =/= -31  | 31 =/= 0    | 0 =/= 31
MTG 1.2  | [12, 34], [-3, 8]   | [-3, 8], [12, 34]   | 41 == 41   | 51 == 51   | 41 =/= -41  | ERR =/= ERR | 35 =/= -35  | 41 =/= 0    | 0 =/= 41
MTG 1.3  | [5, 10], [-3, 8]    | [-3, 8], [5, 10]    | 10 == 10   | 20 == 20   | 10 =/= -10  | ERR =/= ERR | 4 =/= -4    | 10 =/= 0    | 0 =/= 10
MTG 1.4  | [12, 34], [0, 0]    | [0, 0], [12, 34]    | 46 == 46   | 46 == 46   | 46 =/= -46  | ERR =/= ERR | 46 =/= -46  | 46 =/= 0    | 0 =/= 46
MTG 1.5  | [-3, 8], [100, 250] | [100, 250], [-3, 8] | 345 == 345 | 355 == 355 | -345 =/= 345| ERR =/= ERR | -339 =/= 339| 0 =/= 345   | 345 =/= 0

MR2: SHIFT INVARIANCE (dist(A, B) == dist(A+c, B+c))
MTG      | Shift c  | SUT Output | M1 Output   | M2 Output  | M6 Output   | M13 Output  | M17 Output     | M18 Output  | M23 Output
-------------------------------------------------------------------------------------------------------------------------------------
MTG 2.1  | c=+10    | 31 == 31   | 61 =/= 101  | 31 == 31   | ERR =/= ERR | 31 == 31    | 400 =/= 1210   | 61 =/= 101  | 31 == 31
MTG 2.2  | c=-25    | 41 == 41   | 51 =/= 49   | 41 == 41   | ERR =/= ERR | 35 =/= -23  | 308 =/= 517    | 57 =/= 67   | 41 == 41
MTG 2.3  | c=+100   | 15 == 15   | 15 =/= 415  | 15 == 15   | ERR =/= ERR | 15 == 15    | 0 =/= 21500    | 15 =/= 415  | 15 == 15
MTG 2.4  | c=+3.5   | 345 == 345 | 355 =/= 369 | -345 == -345| ERR =/= ERR| -339 =/= -345| 2300 =/= 2967 | 361 =/= 369 | ERR =/= ERR
```

Key Demonstration Takeaways

- Execution Command: python TEST\test.py — completely automated metamorphic & mutation evaluation suite.
- Original SUT Compliance: Passed all 10 MTGs perfectly (100% adherence to symmetry and shift invariance).
- Automated Fault Killing: Instantly reports evaluated numeric relations ('==' vs '=/=') across all 30 mutants.

EXPERIMENTAL METRICS & FORMULAS

Task 2.5: Quantitative Evaluation (ONLY E-measure & P-measure)

Metric	MR1: Symmetry	MR2: Shift Invariance	Combined (MR1 U MR2)
Non-Equivalent Mutants Tested	30	30	30
Mutants Killed	11 / 30	12 / 30	15 / 30
E-measure (Effectiveness)	11/30 = 0.3667 (36.67%)	12/30 = 0.4000 (40.00%)	15/30 = 0.5000 (50.00%)
P-measure: MTG Success Rate (Prate)	5/5 = 1.0000 (100.0%)	5/5 = 1.0000 (100.0%)	10/10 = 1.0000 (100.0%)
P-measure: Mean Detection Prob (Pavg)	0.3667 (36.67%)	0.3267 (32.67%)	0.3467 (34.67%)

Metric Definitions (As Mandated by Requirements)

- E-measure (Effectiveness): Quantifies fault-finding capacity = |Killed(MR)| / Total Mutants. Combined reached 50.00% (15/30).
- P-measure Definition 1 (MTG Success Rate): Proportion of MTGs that killed >= 1 mutant. Both MRs achieved 100% (all 10 MTGs killed mutants).
- P-measure Definition 2 (Average Kill Probability): Mean probability that a randomly chosen MTG detects a fault (36.67% MR1, 32.67% MR2).

Task 2.6: Discussion & Empirical Insights

Complementarity of MR1 & MR2

- Mutants Killed by MR1 Only (M2, M24, M25):
- • M2 (no abs): (a-b) = -(b-a) violates symmetry! But under MR2: (a+c)-(b+c) = a-b preserves shift. MR2 alone completely misses M2!
- • M24/M25: Unidirectional filters break symmetry but satisfy translation invariance.
- Mutants Killed by MR2 Only (M1, M17, M18, M23):
- • M1 (a+b): Addition is symmetric (a+b = b+a) -> misses MR1! But under MR2: (a+c)+(b+c) = a+b+2c != a+b -> killed by MR2!
- • M23: Restricted to int -> float shift c=3.5 triggers TypeError.
- Mutants Killed by Both (8 mutants): M6, M13, M19, M26-M30.

Analysis of 15 Surviving Mutants

- Why Did 15 Mutants Survive?
- • Metric Invariance Preservation: Mutants like M12 ((a-b)^2) preserve both symmetry ((-x)^2 = x^2) and shift invariance (((a+c)-(b+c))^2 = (a-b)^2).
- • Global Scaling: M10 (*2.0) preserves both properties.
- • Constant Biases: M8 (+1.0) and M9 (-1.0) satisfy both.
- Key Academic Reflection:
- • No single MR is a silver bullet.
- • Metamorphic testing requires multiple orthogonal MRs (e.g., adding Homogeneity dist(kA, kB) = |k|*dist(A, B) and Triangle Inequality) to reach near-100% kill rate.

Project Summary & APA 7 References

Submission Package & Checklist

- Delivered Files & Folders:
- • ProjectReport-[ID]-[Name].pdf (<= 12 pages, 12pt Times, A4)
- • Presentation_Slides.pdf (Widescreen 16:9 PDF deck)
- • ProjectReport-[ID]-[Name].zip containing:
- - SUT/ (manhattan_distance.py, readme.txt)
- - MUTANTS/ (mutant1.py ... mutant30.py)
- - TEST/ (mr1.py, mr2.py, test.py)
- Compliance Verification:
- • 2D vectors strictly used: T1=[12,34], T2=[5,10]...
- • ONLY E-measure and P-measure evaluated
- • All files < 10 MB limit (ZIP 22.9 KB, PDF 541 KB)

References (APA 7th Edition)

- Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., & Yoo, S. (2015). The oracle problem in software testing: A survey. IEEE TSE, 41(5), 507–525.
- Chen, T. Y., Cheung, S. C., & Yiu, S. M. (1998). Metamorphic testing: a new approach for generating next test cases. HKUST-CS98-01.
- Chen, T. Y., Kuo, F.-C., Liu, H., et al. (2018). Metamorphic testing: A review of challenges and opportunities. ACM Computing Surveys, 51(1), Article 4.
- Chen, T. Y., Tse, T. H., & Zhou, Z. Q. (2004). Fault-based testing without the need of oracles. Inf. Softw. Technol., 46(7), 449–460.
- Le, V., Afshari, M., & Su, Z. (2014). Compiler validation via equivalence modulo inputs. ACM SIGPLAN Notices, 49(6), 216–226.
- Segura, S., et al. (2016). A survey on metamorphic testing. IEEE TSE, 42(9), 805–824.
- TheAlgorithms. (2026). Manhattan distance. GitHub.
- Weyuker, E. J. (1982). On testing non-testable programs. The Computer Journal, 25(4), 465–470.